

PROVA DE APTIDÃO PROFISSIONAL



CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

NOME ALUNO: Bernardo Ângelo Santos

N.º: 6

PROFESSOR ORIENTADOR: Luís Fernandes

ANO LETIVO: 2025/2026



DEDICATÓRIA

Dedico este relatório a todos aqueles que contribuíram para o meu crescimento pessoal e académico, em especial à minha família, cujo apoio foi fundamental para a concretização deste projeto.



TGP SI
CURSO PROFISSIONAL

 **REPÚBLICA
PORTUGUESA**

EDUCAÇÃO



Os Fundos Europeus mais próximos de si.



**SELO DE
CONFORMIDADE
EQAVET**



AGRADECIMENTOS

Agradeço aos meus professores pelo acompanhamento constante, pela orientação ao longo de todo o projeto e pelos conhecimentos transmitidos durante estes três anos. Aos meus colegas, pela entajuda, partilha de ideias e espírito de colaboração que tornaram este percurso mais leve e motivador. À minha família, deixo um agradecimento especial pelo apoio incondicional, pela paciência e por acreditarem sempre em mim. Sem o vosso incentivo, este trabalho não teria sido possível.



TGP SI
CURSO PROFISSIONAL

 **REPÚBLICA
PORTUGUESA**

educação

 **PESSOAS
2030**

Os Fundos Europeus mais próximos de si.

 **PORTUGAL
2030**

 **Cofinanciado pela
União Europeia**



**SELO DE
CONFORMIDADE
EQAVET**



ÍNDICE

Dedicatória.....	I
Agradecimentos	II
Índice	III
Índice de figuras	V
Índice de tabelas	VI
Lista de siglas e acrónimos	VII
1. Introdução.....	1
1.1. Fundamentação.....	2
1.2. Descrição do projeto	3
1.3. Objetivos do projeto	4
1.4. Estado do conhecimento.....	5
2. Planificação do projeto	6
2.1. Calendarização.....	6
2.1.1. Calendarização prevista	7
2.1.2. Calendarização realizada	8
2.2. Tecnologia utilizada.....	9
2.3. Protótipo	13
2.4. Base de Dados	14
3. Implementação do projeto	16
3.1 Preparação do ambiente de desenvolvimento	16
3.2 Configuração Inicial	16
3.3 Estrutura do Projeto	18
3.4 Desenvolvimento da base de dados	18
3.5 Desenvolvimento da Autenticação	19
3.6 Sistema de Permissões	20
3.7 Desenvolvimento das Funcionalidades Principais	20
3.8 Desenvolvimento do BackOffice	22
3.9 Definição das Rotas	23
3.10 Controladores.....	25



TGPSP
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

Educação

PESSOAS
2030

PORTUGAL
2030

Cofinanciado pela
União Europeia



Os Fundos Europeus mais próximos de si.

3.11 Views	27
3.12 Gestão das Imagens	28
3.13 Validação dos Dados.....	29
3.14 Testes realizados.....	30
4. Problemas e soluções encontradas	31
5. Conclusão	32
Referências bibliográficas	33

ÍNDICE DE FIGURAS

Figura 1 - Logo do Visual Studio Code	9
Figura 2 - Logo do Laragon	9
Figura 3 - Logo do Laravel	9
Figura 4 - Logo do php	10
Figura 5 - Logo do Composer	10
Figura 6 - Logo do HTML5	10
Figura 7 - Logo do CSS3	11
Figura 8 - Logo do JavaScript	11
Figura 9 - Logo do MySQL Workbench	11
Figura 10 - Logo do Mailtrap	12
Figura 11 - Logo do Github	12
Figura 12 - Logo do Github Copilot	12
Figura 13 - Protótipo	13
Figura 14 - Base de Dados	15
Figura 15 - Configuração da base de dados no .env	16
Figura 16 - Configuração do mailtrap no .env	17
Figura 17 - Backoffice (Visão Geral)	23
Figura 18 - Definição das Rotas	24
Figura 19 - Página de Favoritos	25
Figura 20 - Página de Definições da Conta	26
Figura 21 - Página de Login	26
Figura 22 - Backoffice (Gestão do Catálogo de Veículos)	27
Figura 23 - Problema com os Recursos Visuais do Site	31

ÍNDICE DE TABELAS

Tabela 1 - Cronograma inicial	7
Tabela 2 - Cronograma final	8



LISTA DE SIGLAS E ACRÓNIMOS

PAP – Prova de Aptidão Profissional



TGP SI
CURSO PROFISSIONAL

 **REPÚBLICA
PORTUGUESA**

EDUCAÇÃO

 **PESSOAS
2030**

 **PORTUGAL
2030**

 **Cofinanciado pela
União Europeia**

Os Fundos Europeus mais próximos de si.



**SELO DE
CONFORMIDADE
EQAVET**

1. INTRODUÇÃO

A PAP consiste na apresentação e defesa, perante um júri, de um projeto consubstanciado num produto, material ou intelectual, numa intervenção ou numa atuação, consoante a natureza dos cursos, bem como no respetivo relatório final de realização e apreciação crítica, demonstrativo de conhecimentos, aptidões, atitudes e competências profissionais adquiridas ao longo do percurso formativo do aluno, em todas as componentes de formação, com especial enfoque nas áreas de competência inscritas no Perfil dos Alunos à Saída da Escolaridade Obrigatória e no perfil profissional associado à respetiva qualificação.

O presente relatório relata as diversas etapas do desenvolvimento do projeto realizado no âmbito da PAP. Está dividido em cinco capítulos. O primeiro é esta introdução, onde são apresentadas as razões que levaram ao desenvolvimento deste projeto, bem como a descrição e os objetivos do mesmo. O segundo capítulo corresponde à Fundamentação Teórica, reunindo a pesquisa realizada sobre o tema, o estado do conhecimento e as tecnologias selecionadas para o desenvolvimento da BrebasMotors. O terceiro capítulo apresenta a Planificação do Projeto, incluindo a calendarização prevista e realizada, bem como a organização das etapas de trabalho. O quarto capítulo descreve a Implementação, detalhando o processo de construção da plataforma, desde a base de dados até à programação das funcionalidades. Por fim, o quinto capítulo refere-se à Avaliação Crítica, onde são analisados os resultados obtidos, as dificuldades encontradas e uma reflexão final sobre o trabalho desenvolvido.

A escolha deste projeto foi motivada não só pela relevância crescente da digitalização no setor automóvel, mas também por um gosto pessoal pelo mundo automóvel, desenvolvido ao longo dos anos. O interesse por veículos, pelas suas características técnicas e pela forma como são apresentados ao público, tornou evidente a oportunidade de criar uma plataforma que reunisse estas duas áreas: tecnologia e automóveis. Assim, a BrebasMotors surgiu como uma forma de unir este interesse pessoal com a aplicação prática dos conhecimentos adquiridos ao longo do curso, permitindo desenvolver um projeto útil, atual e alinhado com as exigências do mercado.

O projeto consiste no desenvolvimento da BrebasMotors, uma plataforma digital dedicada à apresentação e gestão de veículos de uma concessionária automóvel. A plataforma integra uma área pública, onde os utilizadores podem pesquisar automóveis através de filtros avançados, consultar detalhes completos e guardar favoritos, e uma área de BackOffice destinada à administração do catálogo.

Os principais objetivos do projeto passam por facilitar a pesquisa de veículos, assegurar informação rigorosa e atualizada, melhorar a experiência de navegação, otimizar a gestão interna da concessionária e demonstrar competências técnicas adquiridas ao longo do percurso formativo.



Com esta estrutura, a BrebasMotors apresenta-se como uma solução digital completa, alinhada com os objetivos da PAP e com as exigências atuais do mercado automóvel, demonstrando competências técnicas, organizacionais e de desenvolvimento adquiridas ao longo do curso.

1.1. FUNDAMENTAÇÃO

A escolha deste tema surgiu de forma natural devido ao meu gosto pessoal pelo setor automóvel, aliado ao interesse em desenvolver soluções digitais que melhorem a experiência dos utilizadores. Desde cedo que acompanho a evolução do setor, tanto ao nível dos veículos como das plataformas online utilizadas pelas concessionárias, o que me permitiu identificar várias limitações presentes em muitos websites atuais. Assim, a criação da BrebasMotors representa a oportunidade de unir este interesse pessoal com a aplicação prática dos conhecimentos adquiridos ao longo do curso, desenvolvendo uma plataforma moderna, funcional e capaz de responder às necessidades reais de uma concessionária automóvel.

A criação da BrebasMotors surge da necessidade crescente de modernização e digitalização no setor automóvel, onde a maioria dos consumidores inicia a sua pesquisa de veículos através da internet. Apesar da evolução tecnológica, muitos websites de concessionárias continuam a apresentar limitações significativas, como interfaces pouco intuitivas, ausência de filtros eficientes ou falta de ligação a bases de dados atualizadas, dificultando a experiência do utilizador e a gestão interna das empresas.

A BrebasMotors foi concebida precisamente para colmatar estas lacunas, oferecendo uma plataforma moderna, organizada e funcional que permite ao utilizador procurar veículos de forma rápida e personalizada. A integração de filtros avançados, como marca, modelo, segmento, combustível e quilometragem, associados a uma base de dados externa, garante resultados precisos e atualizados, facilitando a tomada de decisão e reduzindo o tempo de pesquisa.

Além da vertente pública, o projeto inclui uma área de BackOffice exclusiva a administradores, permitindo gerir o catálogo de veículos de forma eficiente, segura e profissional.

A relevância da BrebasMotors assenta na sua capacidade de oferecer uma solução digital robusta, intuitiva e alinhada com as exigências atuais do mercado automóvel. Ao combinar utilidade, organização e funcionalidades avançadas, o projeto posiciona-se como uma alternativa moderna e diferente, contribuindo para melhorar a experiência de compra dos utilizadores e otimizar os processos internos da concessionária.



TGPSP
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

Educação

PESSOAS
2030

PORTUGAL
2030

Os Fundos Europeus são próximos de si.

Cofinanciado pela
União Europeia

SELO DE
CONFORMIDADE
EQAVET

1.2. DESCRIÇÃO DO PROJETO

A BrebasMotors é uma plataforma digital desenvolvida com o objetivo de modernizar a forma como uma concessionária automóvel apresenta, organiza e gere o seu catálogo de veículos. O projeto foi concebido para oferecer uma solução robusta, intuitiva e visualmente apelativa, capaz de responder às necessidades tanto dos utilizadores comuns como da gestão interna da concessionária.

Na área pública do website, os utilizadores podem explorar uma vasta lista de automóveis através de filtros avançados, como marca, modelo, segmento, tipo de combustível, quilometragem e estado do veículo. Estes filtros estão diretamente ligados a uma base de dados externa, garantindo que toda a informação apresentada é atualizada, rigorosa e organizada. Cada veículo possui uma página individual com fotografias de alta qualidade, características técnicas, preço e detalhes relevantes, permitindo ao utilizador analisar cada opção de forma clara e completa.

A BrebasMotors inclui também funcionalidades que melhoram a experiência de navegação, como a possibilidade de guardar veículos como favoritos, facilitando a comparação e o acompanhamento de alterações de preço ou disponibilidade. O utilizador dispõe ainda de uma área pessoal onde pode atualizar os seus dados, consultar os seus favoritos e gerir a sua conta.

A componente administrativa do projeto é assegurada pelo BackOffice, uma área exclusiva para utilizadores com permissões especiais. Este painel permite gerir todo o catálogo de veículos, incluindo adicionar novos automóveis, editar informações existentes, remover veículos ou definir o estado de disponibilidade (disponível, reservado ou vendido). Importa destacar que apenas o administrador com o ID 1 possui a capacidade de atribuir permissões de administração a outros utilizadores, garantindo um controlo rigoroso e seguro sobre o acesso às funcionalidades internas.

O BackOffice foi desenvolvido com foco na eficiência, na organização e na segurança, integrando mecanismos de autenticação e controlo de acessos. A BrebasMotors distingue-se pela sua estrutura sólida, pela integração entre FrontEnd e BackEnd e pela preocupação com a experiência do utilizador. O design responsivo garante que o website funciona corretamente em computadores, tablets e smartphones, proporcionando uma navegação fluida em qualquer dispositivo.

Em conjunto, estas funcionalidades tornam a BrebasMotors uma solução digital completa, capaz de melhorar significativamente a forma como uma concessionária apresenta os seus veículos e interage com os seus clientes, enquanto otimiza os processos internos de gestão.



TGPSP
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

Associação

PESSOAS
2030

PORTUGAL
2030

Cofinanciado pela
União Europeia

SELO DE
CONFORMIDADE
EQAVET

Os Fundos Europeus mais próximos de si.

1.3. OBJETIVOS DO PROJETO

O principal objetivo da BrebasMotors é disponibilizar uma plataforma digital moderna, intuitiva e funcional que centralize toda a informação relativa aos veículos de uma concessionária automóvel, permitindo aos utilizadores encontrar rapidamente o automóvel que melhor corresponde às suas preferências. Para isso, a plataforma integra filtros avançados, como marca, modelo, segmento, tipo de combustível, quilometragem e estado do automóvel, garantindo uma pesquisa eficiente, precisa e sempre atualizada graças à ligação direta a uma base de dados externa.

Além de facilitar a pesquisa, a BrebasMotors procura assegurar transparência e rigor na apresentação da informação, oferecendo uma experiência de navegação acessível e responsiva, adaptada a computadores, tablets e smartphones. A plataforma promove também a interatividade, permitindo aos utilizadores guardar veículos como favoritos, consultar detalhes completos de cada automóvel e gerir os seus próprios dados. Paralelamente, inclui uma área de BackOffice segura e funcional que possibilita a gestão interna do catálogo da concessionária, permitindo adicionar, editar, remover e alterar a disponibilidade dos veículos de forma eficiente.

Com estes objetivos, a BrebasMotors pretende melhorar significativamente a experiência digital dos utilizadores, otimizar os processos internos da concessionária e afirmar-se como uma solução tecnológica moderna, organizada e diferente no setor automóvel.



TGP SI
INSTITUTO DE GESTÃO E PLANEAMENTO DA INFORMAÇÃO

REPÚBLICA
PORTUGUESA

educação

PESSOAS
2030

PORTUGAL
2030

Cofinanciado pela
União Europeia

SELO DE
CONFORMIDADE
EQAVET

Os Fundos Europeus moldam o futuro de si.

1.4. ESTADO DO CONHECIMENTO

Antes de iniciar o desenvolvimento do projeto, é essencial compreender alguns conceitos fundamentais relacionados com o setor automóvel e com a criação de sistemas de informação baseados na web. Para garantir que a plataforma cumpre os seus objetivos, torna-se necessário conhecer as características principais dos veículos de alto padrão, bem como os critérios que influenciam a sua pesquisa e seleção, como marca, modelo, segmento, combustível e outras especificações técnicas.

É igualmente importante entender o funcionamento de bases de dados e a forma como estas se relacionam com interfaces web. A ligação entre o website e uma base de dados externa exige conhecimentos sobre armazenamento, atualização e consulta de informação, assegurando que os dados apresentados ao utilizador sejam fiáveis, consistentes e permanentemente atualizados.

Outro aspeto relevante é a compreensão de princípios de usabilidade e acessibilidade que garantem que o website seja intuitivo, fácil de navegar e acessível em diferentes dispositivos. Conceitos como design responsivo, organização visual, clareza da informação e experiência do utilizador, são fundamentais para criar uma plataforma eficiente e agradável de utilizar.

Por fim, é necessário ter noções de segurança da informação, especialmente no que diz respeito ao acesso ao backoffice administrativo. A implementação de mecanismos de autenticação, gestão de permissões e proteção de dados é essencial para garantir que apenas utilizadores autorizados possam adicionar, editar e remover automóveis da base de dados.

Este conjunto de conhecimentos permite definir com precisão os requisitos do sistema e desenvolver um website robusto, seguro e funcional, capaz de responder às necessidades tanto dos utilizadores finais como da própria concessionária.

2. PLANIFICAÇÃO DO PROJETO

O planeamento inicial foi essencial para organizar de forma estruturada todas as fases de desenvolvimento do projeto. Nesta etapa foram definidas as tarefas principais, os prazos e os objetivos de cada fase, garantindo uma sequência lógica de trabalho que permitisse cumprir o calendário estipulado para a PAP. Além disso, foram selecionadas as tecnologias a utilizar e planeada a estrutura geral da aplicação, assegurando que todo o processo de implementação decorreria de forma coerente e eficiente. As secções seguintes apresentam a calendarização prevista e realizada, as ferramentas utilizadas, o protótipo desenvolvido e a estrutura da base de dados que serviu de base ao projeto.

2.1. CALENDARIZAÇÃO

Foi elaborada uma calendarização no início do projeto com o objetivo de organizar todas as etapas de desenvolvimento. Esta fase permitiu definir uma sequência lógica de tarefas, distribuindo-as ao longo dos meses de forma equilibrada e garantindo que o trabalho seria concluído dentro do prazo estipulado para a PAP.



TGP SI
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

Educação

PESSOAS
2030

PORTUGAL
2030

Cofinanciado pela
União Europeia

SELO DE
CONFORMIDADE
EQAVET

Os Fundos Europeus mais próximos de si.

2.1.1. CALENDARIZAÇÃO PREVISTA

O cronograma inicial contemplou diversas etapas desenvolvimento, servindo como guia para orientar o progresso do projeto desde o planeamento até à implementação.

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Pesquisa sobre o tema									
Definição dos Objetivos e Funcionalidades									
Definição do nome e desenho do logótipo									
Desenho do Protótipo									
Preparação do Ambiente de Desenvolvimento									
Desenvolvimento da Base de Dados									
Programação									
Testes e Ajustes Finais									
Lançamento do Projeto Final									
Apresentação do Projeto Final									

Tabela 1 - Cronograma inicial

2.1.2. CALENDARIZAÇÃO REALIZADA

A calendarização realizada apresenta algumas diferenças em relação ao planeamento inicial, devido a imprevistos que surgiram durante o desenvolvimento, como a avaria do computador da escola e a perda total do projeto numa fase inicial. Estes acontecimentos obrigaram a reajustar prazos e redistribuir tarefas ao longo dos meses seguintes.

Apesar dos contratempos, todas as etapas essenciais foram concluídas, ainda que com uma duração diferente da prevista. A calendarização final reflete uma adaptação natural às dificuldades encontradas e demonstra a capacidade de reorganizar o trabalho para garantir a conclusão da BrebasMotors dentro do período definido para a PAP.

ETAPAS	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN	JUL
Pesquisa sobre o tema									
Definição dos Objetivos e Funcionalidades									
Definição do nome e desenho do logótipo									
Desenho do Protótipo									
Preparação do Ambiente de Desenvolvimento									
Desenvolvimento da Base de Dados									
Programação									
Testes e Ajustes Finais									
Lançamento do Projeto Final									
Apresentação do Projeto Final									

Tabela 2 - Cronograma final

2.2. TECNOLOGIA UTILIZADA

Durante o desenvolvimento do projeto foram utilizadas diversas tecnologias, ferramentas e ambientes de trabalho que permitiram garantir a construção de uma plataforma moderna, funcional e segura. Entre as principais tecnologias destacam-se:

- Visual Studio Code – Editor de código utilizado para programar todo o projeto, pela sua leveza, extensões úteis e integração com o Github.

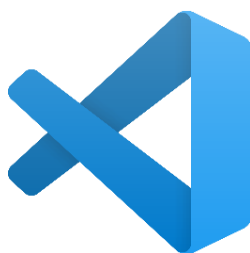


Figura 1 - Logo do Visual Studio Code

- Laragon – Programa responsável pela ligação do projeto à base de dados MySQL.



Figura 2 - Logo do Laragon

- Laravel (PHP Framework) – Framework utilizada para desenvolver o website, garantindo segurança, organização e rapidez no desenvolvimento.

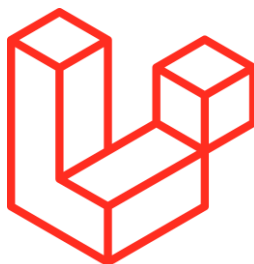


Figura 3 - Logo do Laravel

- PHP – Linguagem responsável pela lógica do servidor, comunicação com a base de dados e implementação das funcionalidades do BackOffice.



Figura 4 - Logo do php

- Composer – Ferramenta de gestão de dependências php, usada para instalar e atualizar bibliotecas essenciais ao Laravel. Simplifica a configuração do BackEnd e automatiza a integração de pacotes externos.



Figura 5 - Logo do Composer

- HTML5 – Utilizado para estruturar todas as páginas do website.



Figura 6 - Logo do HTML5

- CSS3 – Responsável pela estilização, design responsivo e apresentação visual da plataforma.



Figura 7 - Logo do CSS3

- JavaScript – Implementado para melhorar a interatividade do website, dinamizar elementos e otimizar a experiência do utilizador.



Figura 8 - Logo do JavaScript

- MySQL Workbench – Sistema de gestão de base de dados utilizado para armazenar toda a informação relativa aos automóveis e utilizadores.



Figura 9 - Logo do MySQL Workbench

- Mailtrap – Serviço utilizado para testar o envio de emails, garantindo que funcionalidades como recuperação de password funcionam corretamente sem risco de envio real.



Figura 10 - Logo do Mailtrap

- GitHub – Plataforma utilizada para armazenamento do código e criação de backups do projeto.



Figura 11 - Logo do Github

- GitHub Copilot – Assistente de programação que ajudou na escrita de código mais eficiente e na resolução de problemas.



Figura 12 - Logo do Github Copilot

2.3. PROTÓTIPO

Antes de avançar para a implementação da BrebasMotors, tornou-se essencial desenvolver um protótipo que permitisse visualizar a estrutura geral da plataforma e definir a organização das suas principais páginas. Esta etapa foi fundamental para validar a disposição dos elementos, testar a navegabilidade e garantir que a experiência do utilizador seria intuitiva e funcional. O protótipo serviu também como guia para o desenvolvimento técnico, facilitando a escolha das tecnologias, a definição das funcionalidades e a construção da interface final da aplicação.

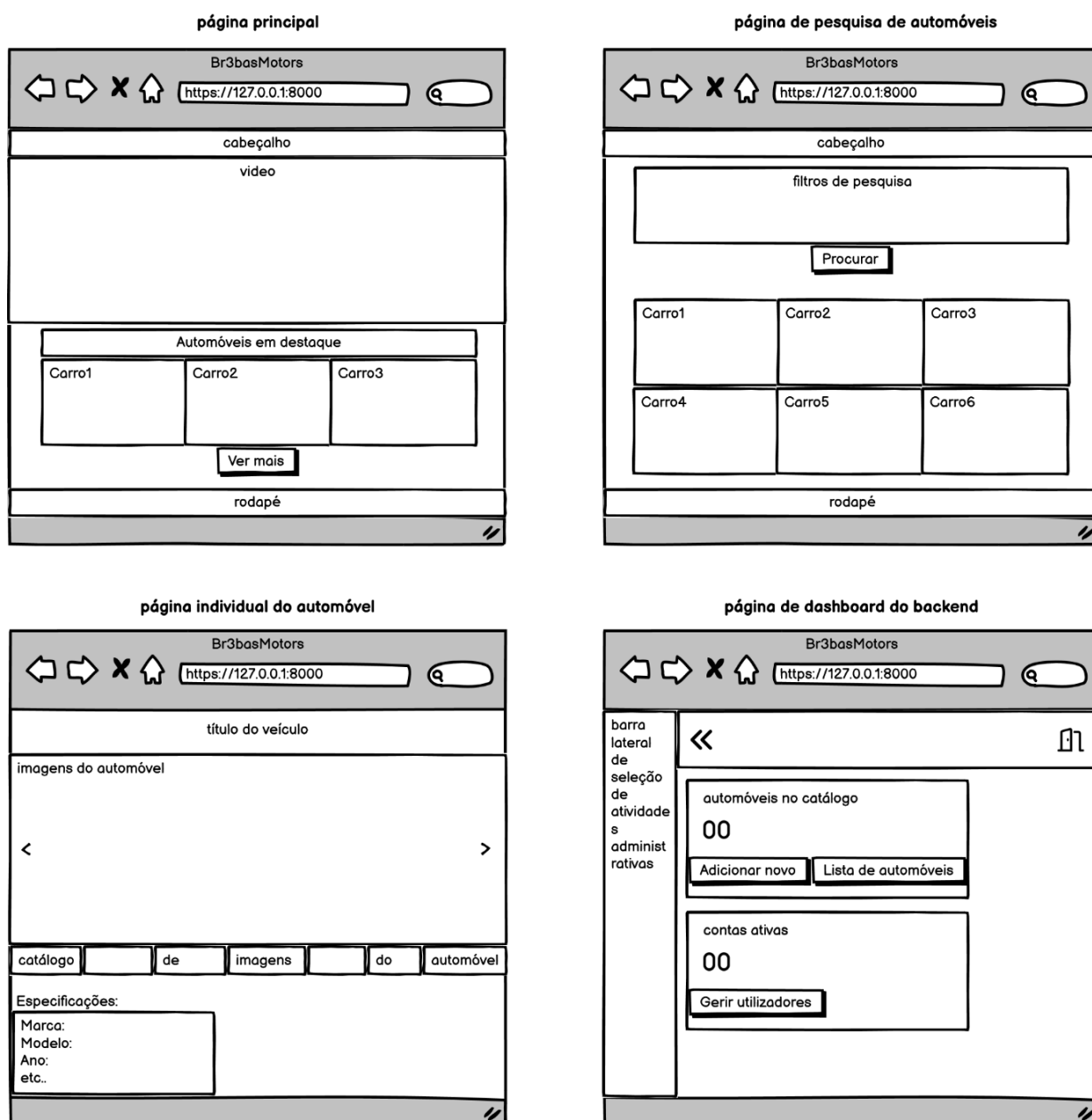


Figura 13 - Protótipo

2.4. BASE DE DADOS

A base de dados da BrebasMotors constitui o núcleo da aplicação, garantindo a organização, integridade e ligação entre toda a informação relativa aos veículos e aos utilizadores. O modelo foi estruturado de forma relacional, permitindo que cada tabela desempenhe uma função específica e se articule com as restantes através de chaves primárias e estrangeiras.

A tabela cars é a principal tabela do sistema, reunindo todos os dados dos automóveis, como marca, modelo, segmento, tipo de combustível, transmissão, quilometragem, estado e imagens. Cada veículo está associado às tabelas brands, car_models, segments, fuels e transmissions, que funcionam como tabelas de catálogo normalizado. Estas tabelas garantem consistência na informação, evitando duplicações e permitindo que cada automóvel referencie diretamente os seus atributos através de relações *um-para-muitos*.

A tabela favorite_cars estabelece a ligação entre os utilizadores e os veículos que marcaram como favoritos, representando uma relação *muitos-para-muitos* entre as tabelas users e cars. Cada registo identifica um utilizador e o respetivo automóvel selecionado, permitindo personalizar a experiência de navegação.

A tabela tipo define os perfis de utilizador, distinguindo entre Admin e User, e estabelece uma relação direta com a tabela users. Esta estrutura permite controlar permissões, garantindo que apenas administradores têm acesso às funcionalidades de gestão do BackOffice, como adicionar, editar ou remover veículos.

Em conjunto, estas tabelas formam uma base de dados coerente, organizada e otimizada, assegurando que toda a informação da plataforma é armazenada de forma estruturada e que as relações entre veículos, utilizadores e categorias funcionam corretamente ao longo de toda a aplicação.

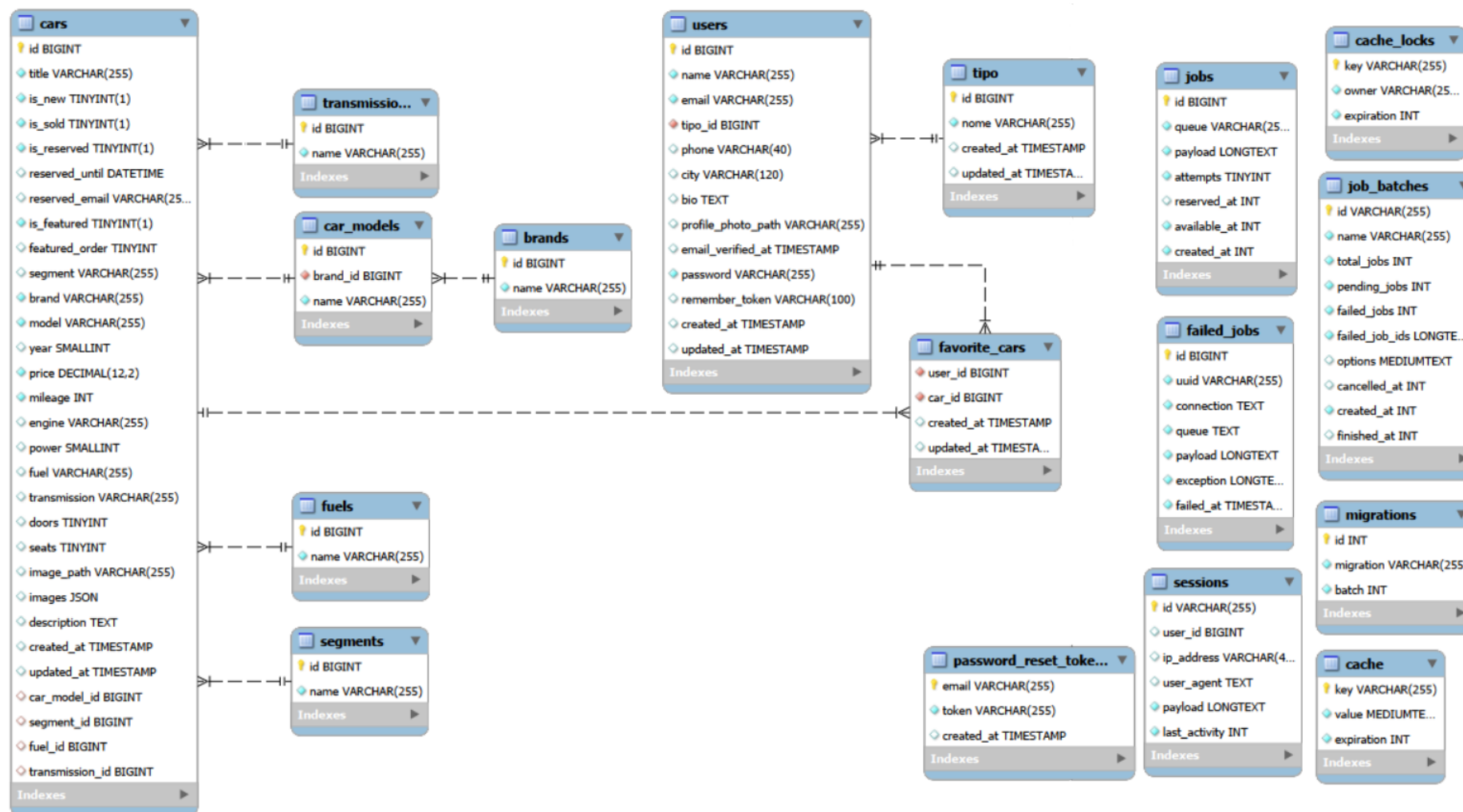


Figura 14 - Base de Dados

3. IMPLEMENTAÇÃO DO PROJETO

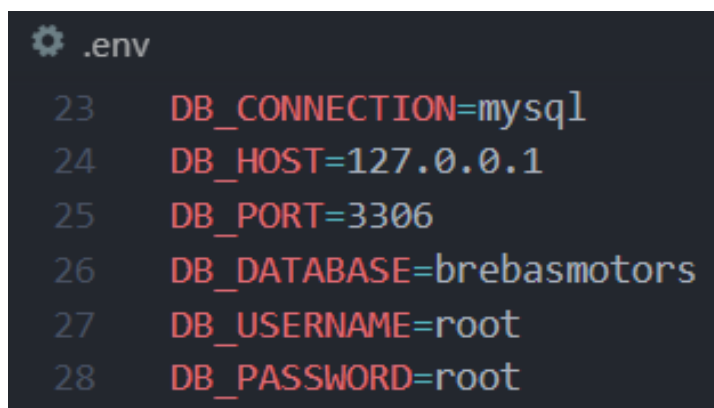
3.1 PREPARAÇÃO DO AMBIENTE DE DESENVOLVIMENTO

A preparação do ambiente de desenvolvimento foi realizada com o objetivo de garantir que todas as ferramentas necessárias ao projeto estavam corretamente instaladas e configuradas. Para isso, procedi à instalação do Laravel 12, suportado pelo PHP 8.2, através do Composer, que permitiu gerir dependências e estruturar o backend da aplicação. Em seguida, configurei o npm para tratar dos componentes do frontend, instalando e integrando ferramentas como Vite, Tailwind CSS, Axios e o laravel-vite-plugin, essenciais para a construção de interfaces modernas, responsivas e eficientes. Todo o desenvolvimento foi efetuado no Visual Studio Code, onde utilizei extensões específicas para PHP, Blade, Laravel e Tailwind CSS, de forma a otimizar a escrita do código e melhorar a produtividade. Esta preparação assegurou um ambiente estável, organizado e adequado às necessidades técnicas da BrebasMotors.

3.2 CONFIGURAÇÃO INICIAL

A configuração inicial do projeto começa pela definição correta das variáveis de ambiente no ficheiro `.env`, garantindo que a aplicação comunica com os serviços necessários. Entre os parâmetros essenciais estão:

- `APP_NAME`, `APP_ENV`, `APP_URL` — identificação da aplicação e ambiente de execução.
- `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD` — credenciais e configuração da base de dados.
- `MAIL_MAILER`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD` — dados do serviço de email usado para envio de notificações.



```
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=brebasmotors
27 DB_USERNAME=root
28 DB_PASSWORD=root
```

Figura 15 - Configuração da base de dados no `.env`

Ligação à Base de Dados

A estrutura da base de dados é gerida através das *migrations* do Laravel, que criam automaticamente as tabelas principais. Comando principal: php artisan migrate

Configuração do Armazenamento

O projeto utiliza o disco public para guardar imagens e outros ficheiros acessíveis externamente. Para ativar o acesso público, é necessário executar o comando: php artisan storage:link

Envio de Emails

A aplicação envia notificações relacionadas com reservas, preços e vendas, recorrendo ao sistema de filas do Laravel (ShouldQueue) para processamento assíncrono. Para que o envio seja funcional, é obrigatório configurar um driver SMTP ou SES no ficheiro .env, permitindo que os emails sejam enviados em produção.

```
50 MAIL_MAILER=smtp
51 MAIL_SCHEME=smtp
52 MAIL_HOST=sandbox.smtp.mailtrap.io
53 MAIL_PORT=2525
54 MAIL_USERNAME=7e0eb9e01acf1d
55 MAIL_PASSWORD=b25aee1f38bacf
56 MAIL_FROM_ADDRESS="support@brebasmotors.com"
57 MAIL_FROM_NAME="Br3basMotors"
58
```

Figura 16 - Configuração do mailtrap no .env

3.3 ESTRUTURA DO PROJETO

A estrutura do projeto seguiu o modelo padrão do Laravel, baseado na arquitetura MVC (Model-View-Controller), o que permitiu garantir uma separação lógica entre componentes, maior organização do código e facilidade de manutenção ao longo do desenvolvimento. A organização das pastas e ficheiros foi fundamental para assegurar que cada parte da aplicação desempenhava o seu papel de forma clara e eficiente.

Os Controllers concentraram a lógica da aplicação, sendo responsáveis por processar pedidos, validar dados e devolver respostas adequadas ao utilizador. Os Models, implementados através do Eloquent, representaram as entidades da base de dados, definindo regras, atributos e relacionamentos entre tabelas. As rotas acessíveis via HTTP foram definidas no ficheiro `routes/web.php`, que estabeleceu a ligação entre os pedidos do utilizador e os métodos dos controladores. As interfaces da plataforma foram construídas na pasta `resources/views`, onde os templates Blade permitiram desenvolver o front-end da área pública, da área de utilizador e do BackOffice. Por fim, a pasta `database/migrations` reuniu os ficheiros responsáveis por definir o esquema da base de dados, permitindo criar e alterar tabelas de forma controlada.

A arquitetura MVC revelou-se vantajosa ao longo do projeto, uma vez que proporcionou uma separação clara de responsabilidades, tornando o código mais organizado e fácil de evoluir. A reutilização de componentes foi facilitada, assim como a criação de testes e a manutenção futura da aplicação. Esta estrutura contribuiu para um desenvolvimento mais eficiente, coerente e alinhado com as boas práticas recomendadas pelo Laravel.

3.4 DESENVOLVIMENTO DA BASE DE DADOS

O desenvolvimento da base de dados foi realizado de forma estruturada, começando pela criação das migrações que definiram o esquema fundamental da aplicação. A tabela `cars` foi criada como tabela principal, reunindo toda a informação relativa aos veículos. Em seguida, foi construída a tabela `users`, que incluiu o campo `tipo_id` para distinguir os diferentes perfis de utilizador. Para permitir que os utilizadores marcassem veículos como favoritos, foi desenvolvida a tabela `favorite_cars`, que funcionou como tabela pivot entre utilizadores e automóveis. Posteriormente, foram adicionados novos campos relacionados com o sistema de reservas, como `is_reserved`, `reserved_until` e `reserved_email`, permitindo gerir a disponibilidade dos veículos.

Após a definição das migrações, foram criados os modelos Eloquent correspondentes às entidades da aplicação, nomeadamente `Car`, `User`, `Brand`, `CarModel`, `Segment`, `Fuel`, `Transmission` e `Filter`, garantindo que cada tabela tinha a sua representação lógica no código.

A etapa seguinte consistiu na definição dos relacionamentos entre as entidades. O modelo `Car` foi configurado para pertencer a `CarModel`, `Segment`, `Fuel` e `Transmission`, assegurando que cada veículo estivesse corretamente associado às suas características. Além disso, foi estabelecida uma

relação muitos-para-muitos entre Car e User, através da tabela `favorite_cars`, permitindo que os utilizadores guardassem veículos como favoritos. O modelo User refletiu esta mesma relação, garantindo coerência entre ambas as entidades.

Foram utilizados diferentes tipos de relacionamentos ao longo da modelação. As relações um-para-muitos permitiram, por exemplo, que uma marca tivesse vários modelos (Brand → CarModel) e que um segmento agrupasse vários veículos (Segment → Car). Já as relações muitos-para-muitos foram aplicadas no sistema de favoritos, onde um utilizador podia marcar vários carros e cada carro podia ser favorito de vários utilizadores.

Esta estrutura relacional permitiu criar uma base de dados organizada, coerente e totalmente adaptada às necessidades da BrebasMotors, garantindo integridade dos dados e facilitando a implementação das funcionalidades da plataforma.

3.5 DESENVOLVIMENTO DA AUTENTICAÇÃO

A autenticação da plataforma foi desenvolvida manualmente através do *AuthController*, permitindo controlar de forma personalizada todo o fluxo de registo, login e logout. Durante esta fase, implementei os métodos responsáveis por apresentar os formulários e processar as ações dos utilizadores, garantindo que o sistema validava corretamente as credenciais e iniciava ou terminava sessões de forma segura. O processo de registo incluiu validações específicas, como verificação de email, confirmação de password e associação do utilizador ao respetivo perfil definido na base de dados. O login foi configurado para verificar credenciais, bloquear acessos inválidos e redirecionar o utilizador para a área correspondente ao seu tipo de conta.

A recuperação de palavra-passe foi desenvolvida através dos controladores *ForgotPasswordController* e *ResetPasswordController*, que permitiram enviar emails de recuperação e redefinir credenciais após validação do *token*. Este sistema assegurou que os utilizadores pudessem recuperar o acesso à sua conta de forma segura, seguindo o fluxo padrão de envio de email, validação de link e criação de nova password.

A verificação de email foi igualmente configurada através do *VerificationController*, garantindo que cada conta criada era confirmada antes de aceder às funcionalidades reservadas. Esta etapa foi essencial para evitar contas falsas, reforçar a segurança da plataforma e assegurar que apenas utilizadores legítimos tinham acesso às áreas protegidas.

Todos os formulários e páginas de autenticação foram integrados na pasta *auth* e adaptados ao *layout* geral do site, assegurando consistência visual entre a área pública, a área de utilizador e o BackOffice. Esta integração permitiu que a experiência de navegação fosse uniforme, independentemente da secção da plataforma em que o utilizador se encontrasse.

3.6 SISTEMA DE PERMISSÕES

O sistema de permissões foi implementado sem recurso a pacotes externos, optando por uma abordagem simples e eficiente baseada no campo *tipo_id* da tabela *users*. Este campo definiu o nível de acesso de cada utilizador, distinguindo entre perfis de administrador e utilizador comum. A escolha desta abordagem permitiu manter o controlo total sobre o funcionamento das permissões, evitando dependências externas e garantindo maior flexibilidade na implementação.

Para assegurar que apenas administradores acediam às funcionalidades sensíveis, alguns controladores, como o *BackOfficeController* e o *CarController*, incluíram métodos internos de verificação, como *ensureAdmin()* e *ensureUsersManagementAccess()*. Estes métodos foram responsáveis por validar o perfil do utilizador antes de executar ações críticas, como adicionar, editar ou remover veículos, gerir contas ou alterar permissões. Caso o utilizador não tivesse privilégios suficientes, o sistema bloqueava o acesso e redirecionava para páginas apropriadas, garantindo segurança e integridade dos dados.

A gestão dos cargos foi assegurada através da tabela *tipo*, onde estavam definidos os perfis disponíveis, como administrador e utilizador. A atribuição ou alteração do cargo de um utilizador foi implementada no método *usersUpdateTipo*, permitindo que apenas administradores pudessem promover ou rebaixar contas. Esta funcionalidade foi essencial para controlar o acesso ao BackOffice e garantir que apenas utilizadores autorizados tinham capacidade de gerir a plataforma.

Além disso, todas as rotas do BackOffice e as operações de gestão de veículos foram protegidas, permitindo acesso apenas a administradores autenticados. Esta proteção assegurou que tarefas como criação, edição ou remoção de automóveis permanecessem restritas a utilizadores autorizados, evitando alterações indevidas e garantindo que a plataforma funcione de forma segura e controlada.

3.7 DESENVOLVIMENTO DAS FUNCIONALIDADES PRINCIPAIS

O desenvolvimento das funcionalidades principais da BrebasMotors constituiu uma das fases mais importantes do projeto, uma vez que determinou a forma como os utilizadores e administradores interagem com a plataforma. A gestão de automóveis foi implementada integralmente no *CarController*, onde desenvolvi um sistema CRUD completo que permitiu criar, editar, visualizar e remover veículos do catálogo. Além destas operações fundamentais, implementei também mecanismos que possibilitaram alterar o estado e a disponibilidade de cada automóvel, permitindo classificá-lo como disponível, reservado ou vendido. Esta funcionalidade foi essencial para garantir que a informação apresentada ao utilizador refletia sempre a situação real do veículo.

O sistema de gestão de imagens exigiu um desenvolvimento mais aprofundado, uma vez que cada automóvel pode possuir uma imagem principal e uma galeria adicional. Para isso, criei funções específicas responsáveis por armazenar, remover e normalizar as imagens enviadas. A função

storeUploadedImages tratou do armazenamento das imagens na pasta dedicada aos veículos, enquanto *deleteStoredImage* permitiu remover ficheiros associados a automóveis. A função *normalizImages* foi responsável por organizar a galeria, garantindo que todas as imagens eram apresentadas de forma consistente e ordenada. A estrutura de armazenamento foi definida com dois campos principais: *image_path*, que guardou a imagem principal, e *images*, que armazenou a galeria adicional em formato JSON.

A gestão dos estados do veículo foi igualmente desenvolvida de forma detalhada. Para assinalar um automóvel como vendido, utilizei o campo *is_sold*, enquanto o sistema de reservas foi implementado através dos campos *is_reserved* e *reserved_until*, que permitiram controlar a validade da reserva e impedir que veículos reservados fossem apresentados como disponíveis. Esta lógica assegurou que o catálogo se mantinha atualizado e que os utilizadores tinham acesso a informação rigorosa sobre a disponibilidade dos automóveis.

O sistema de favoritos constituiu outra funcionalidade central da plataforma, permitindo aos utilizadores guardar veículos de interesse para consulta posterior. Esta funcionalidade foi implementada através do *FavoriteController*, onde o método *toggle* permitiu adicionar ou remover um automóvel da lista de favoritos de forma dinâmica. A lista de favoritos ficou acessível na área pessoal do utilizador, em *account/favorites*, garantindo uma navegação mais personalizada. Para melhorar a experiência de utilização, implementei suporte tanto para AJAX como para redirecionamento tradicional, permitindo que a funcionalidade funcionasse de forma fluida em diferentes contextos de navegação.

Além destas funcionalidades, desenvolvi também a gestão de perfil através do *AccountSettingsController*, que permitiu aos utilizadores atualizar os seus dados pessoais, alterar ou remover a fotografia de perfil e eliminar a conta de forma segura, com confirmação por email e palavra-passe. A pesquisa de veículos foi implementada no método *index* do *CarController*, onde integrei um conjunto alargado de filtros, incluindo marca, modelo, segmento, combustível, transmissão, ano, preço, quilometragem, número de portas e lugares, bem como filtros de disponibilidade. A ordenação por preço, ano ou quilometragem permitiu refinar ainda mais os resultados. A seleção dinâmica de marca e modelo contribuiu para uma experiência de pesquisa mais intuitiva, adaptando automaticamente as opções disponíveis conforme a escolha do utilizador.

Com o desenvolvimento destas funcionalidades, a BrebasMotors tornou-se uma plataforma completa, funcional e alinhada com as necessidades reais de uma concessionária automóvel, garantindo uma experiência de utilização moderna, organizada e eficiente.

3.8 DESENVOLVIMENTO DO BACKOFFICE

O BackOffice da BrebasMotors foi desenvolvido como uma área exclusiva para administradores, garantindo que apenas utilizadores com *tipo_id = 1* tinham acesso às funcionalidades de gestão mais sensíveis da plataforma. Esta separação entre área pública e área administrativa permitiu assegurar maior segurança, controlo e organização na gestão dos dados, evitando que utilizadores comuns pudessem alterar informação crítica.

O painel administrativo foi estruturado de forma a oferecer uma navegação clara e dedicada, reunindo num único espaço todas as ferramentas necessárias para gerir o sistema. A página inicial do BackOffice funciona como *dashboard*, apresentando componentes de apoio à administração e facilitando o acesso às secções de gestão de veículos e de utilizadores. Embora existam elementos visuais relacionados com estatísticas, o foco principal do BackOffice permaneceu na administração do catálogo automóvel e na gestão dos perfis dos utilizadores.

A gestão de automóveis foi uma das funcionalidades centrais do BackOffice e ficou acessível através da secção *back/cars*. Nesta área, implementei um conjunto completo de operações que permitiram listar todos os veículos registados, criar novos automóveis, editar informações existentes e remover entradas do catálogo. Além disso, desenvolvi um sistema de destaques (*highlights*), que possibilitou ordenar manualmente os veículos que deveriam aparecer em posições de maior relevância na página principal. O BackOffice também incluiu ferramentas para gerir reservas, permitindo ao administrador reservar um veículo ou cancelar reservas ativas, assegurando que o estado de cada automóvel se mantinha atualizado e coerente com a realidade da concessionária.

A gestão de utilizadores foi implementada na secção *back/users*, onde é apresentada a lista completa de contas registadas na plataforma. Nesta área, o administrador pode atualizar o cargo de cada utilizador, promovendo ou despromovendo contas conforme necessário. Para garantir total controlo sobre a atribuição de privilégios, apenas o utilizador com ID 1 recebeu permissão para alterar o *tipo_id* de outros utilizadores, evitando que administradores secundários pudessem modificar permissões de forma indevida.

Todas as rotas associadas ao BackOffice foram protegidas através de verificações internas nos controladores, garantindo que apenas administradores autenticados conseguiram aceder às funcionalidades de gestão. Esta abordagem assegurou que operações críticas, como criação, edição ou remoção de veículos, permaneciam restritas a utilizadores autorizados, reforçando a segurança e integridade da plataforma.

Com o desenvolvimento do BackOffice, a BrebasMotors passou a dispor de um sistema administrativo robusto, organizado e funcional, capaz de responder às necessidades reais de uma concessionária automóvel e de garantir uma gestão eficiente tanto de veículos como de utilizadores.

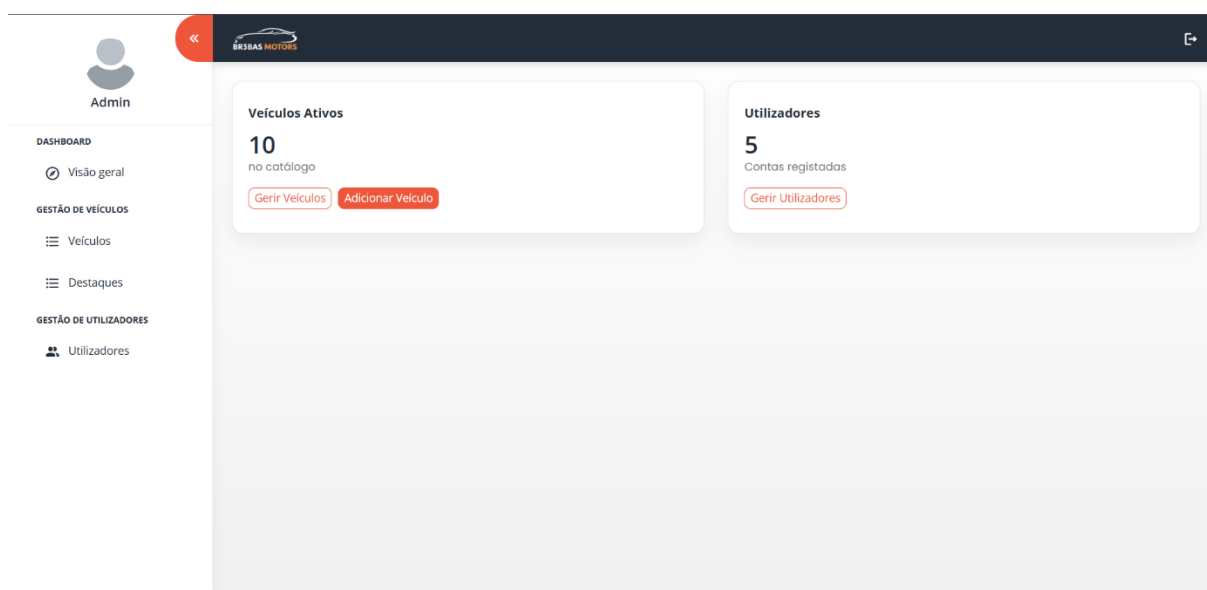


Figura 17 - Backoffice (Visão Geral)

3.9 DEFINIÇÃO DAS ROTAS

A definição das rotas da BrebasMotors foi estruturada de forma a garantir uma separação clara entre o acesso público, o acesso autenticado e as áreas restritas a administradores. Esta organização permitiu controlar de forma rigorosa quem pode aceder a cada parte da aplicação, assegurando segurança, coerência e uma navegação bem definida ao longo de toda a plataforma.

As rotas públicas foram concebidas para serem acessíveis a qualquer utilizador, independentemente de estar autenticado ou não. Nesta categoria inclui a página principal, o catálogo de automóveis, as páginas individuais de cada veículo e ainda secções informativas como contacto, equipa e perguntas frequentes. Estas rotas constituem a base da experiência pública da plataforma, permitindo que qualquer visitante explore os veículos disponíveis e obtenha informação geral sobre a BrebasMotors.

As rotas autenticadas foram desenvolvidas para utilizadores que já efetuaram login, garantindo que apenas contas válidas podem aceder a funcionalidades mais pessoais. Nesta área inclui a gestão de conta, onde o utilizador pode atualizar os seus dados, bem como a lista de favoritos, que reúne os veículos que o utilizador marcou para consulta posterior. A rota responsável por adicionar ou remover favoritos também foi protegida, assegurando que apenas utilizadores autenticados podem interagir com esta funcionalidade.

Por fim, as rotas protegidas por permissões foram reservadas exclusivamente a administradores, definidos pelo campo *tipo_id* = 1. Estas rotas incluem o acesso ao painel administrativo, a gestão completa de automóveis, a gestão de utilizadores, a ordenação de destaques e a funcionalidade de reserva de veículos. Todas estas operações exigem privilégios elevados, uma vez que permitem alterar dados críticos da plataforma. Para reforçar esta segurança, cada rota

administrativa foi validada internamente pelos controladores, garantindo que apenas administradores autenticados e autorizados conseguem executar ações sensíveis.

A organização das rotas foi complementada pela estrutura do ficheiro *web.php*, onde agrupei rotas em blocos distintos, como *guest* para utilizadores não autenticados, *auth* para utilizadores com sessão ativa e secções específicas para rotas administrativas. Esta divisão permitiu manter o código limpo, organizado e fácil de manter, assegurando que cada grupo de rotas refletia claramente o tipo de acesso necessário.

Com esta estrutura, a BrebasMotors passou a dispor de um sistema de navegação robusto, seguro e bem organizado, garantindo que cada utilizador acede apenas às funcionalidades adequadas ao seu perfil e que toda a plataforma funciona de forma coerente e protegida.

```

routes > web.php > ...
50 Route::get('/contact', [ContactController::class, 'show'])->name('contact.show');
51 Route::post('/contact', [ContactController::class, 'send'])->name('contact.send');
52
53 Route::get('/team', function () {
54     return view('team');
55 });
56
57 Route::get('/faq', function () {
58     return view('faq');
59 });
60
61 Route::get('/cars', [CarController::class, 'index'])->name('cars.index');
62 Route::get('/cars/{car}', [CarController::class, 'show'])->name('cars.show');
63
64 // Manual auth routes (login, register, password reset, email verification)
65 Route::middleware('guest')->group(function () {
66     Route::get('login', [AuthController::class, 'showLogin'])->name('login');
67     Route::post('login', [AuthController::class, 'login']);
68
69     Route::get('register', [AuthController::class, 'showRegister'])->name('register');
70     Route::post('register', [AuthController::class, 'register']);
71
72     // Password Reset Routes
73     Route::get('password/reset', [ForgotPasswordController::class, 'showLinkRequestForm'])->name('password.reset');
74     Route::post('password/email', [ForgotPasswordController::class, 'sendResetLinkEmail'])->name('password.reset.request');
75     Route::get('password/reset/{token}', [ResetPasswordController::class, 'showResetForm'])->name('password.reset.show');
76     Route::post('password/reset', [ResetPasswordController::class, 'reset'])->name('password.reset.update');
77 });
78
79 Route::post('logout', [AuthController::class, 'logout'])->name('logout')->middleware('auth');
80
81 Route::middleware('auth')->group(function () {
82     Route::get('settings', [AccountSettingsController::class, 'edit'])->name('account.settings');
83     Route::put('settings', [AccountSettingsController::class, 'update'])->name('account.settings.update');
84     Route::delete('settings', [AccountSettingsController::class, 'destroy'])->name('account.settings.destroy');
85     Route::get('favorites', [FavoriteController::class, 'index'])->name('account.favorites');
86     Route::post('cars/{car}/favorite', [FavoriteController::class, 'toggle'])->name('cars.favorite.toggle');
87     Route::get('back/dashboard', function () {
88         abort_unless((int) Auth::user()->tipo_id === 1, 403);
89     });
90 });

```

Figura 18 – Definição das Rotas

3.10 CONTROLADORES

A aplicação foi estruturada com vários controladores responsáveis por organizar a lógica das diferentes áreas da plataforma. O CarController é o controlador central dedicado à gestão dos veículos, reunindo tanto as funcionalidades públicas como as administrativas. Através dos seus métodos, é possível apresentar a listagem de automóveis, aplicar filtros de pesquisa, criar novos veículos, editar informações existentes, gerir imagens, controlar estados como reserva ou venda e definir veículos em destaque no catálogo. Este controlador assegura que todas as operações relacionadas com automóveis são tratadas de forma consistente e integrada.

O FavoriteController gere o sistema de favoritos, permitindo que cada utilizador adicione ou remova veículos da sua lista pessoal. Este controlador garante uma interação rápida e intuitiva, tanto através de AJAX como por navegação tradicional, oferecendo ao utilizador uma forma simples de acompanhar os veículos que lhe despertam interesse.

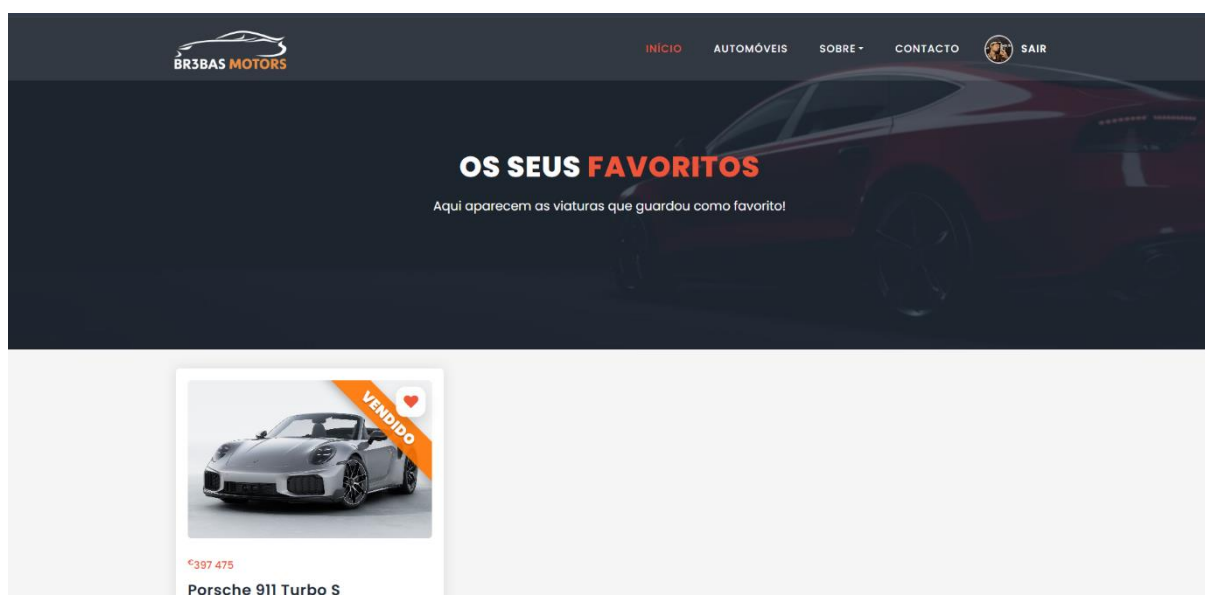


Figura 19 - Página de Favoritos

O AccountSettingsController é responsável pela gestão do perfil do utilizador, disponibilizando funcionalidades para editar dados pessoais, atualizar a fotografia de perfil e eliminar a conta de forma segura. Este controlador assegura que cada utilizador mantém controlo total sobre a sua informação e preferências.

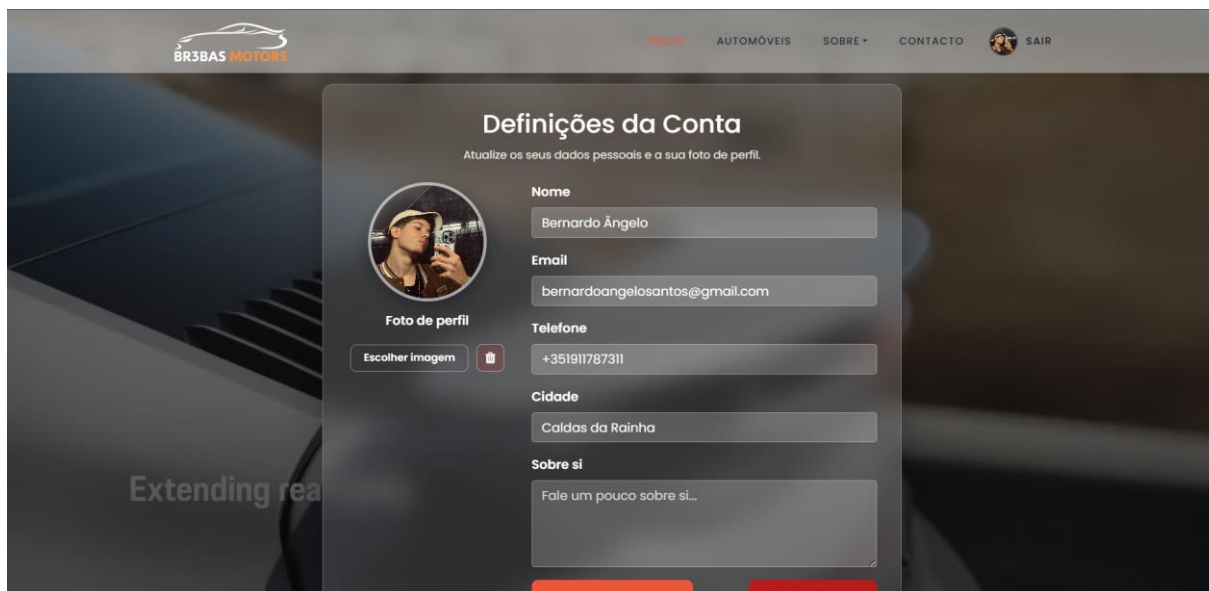


Figura 20 - Página de Definições da Conta

O AuthController concentra todas as operações relacionadas com autenticação, incluindo login, registo, recuperação de palavra-passe e logout. Através deste controlador, a aplicação garante um fluxo de autenticação seguro, validado e adaptado às necessidades da plataforma.

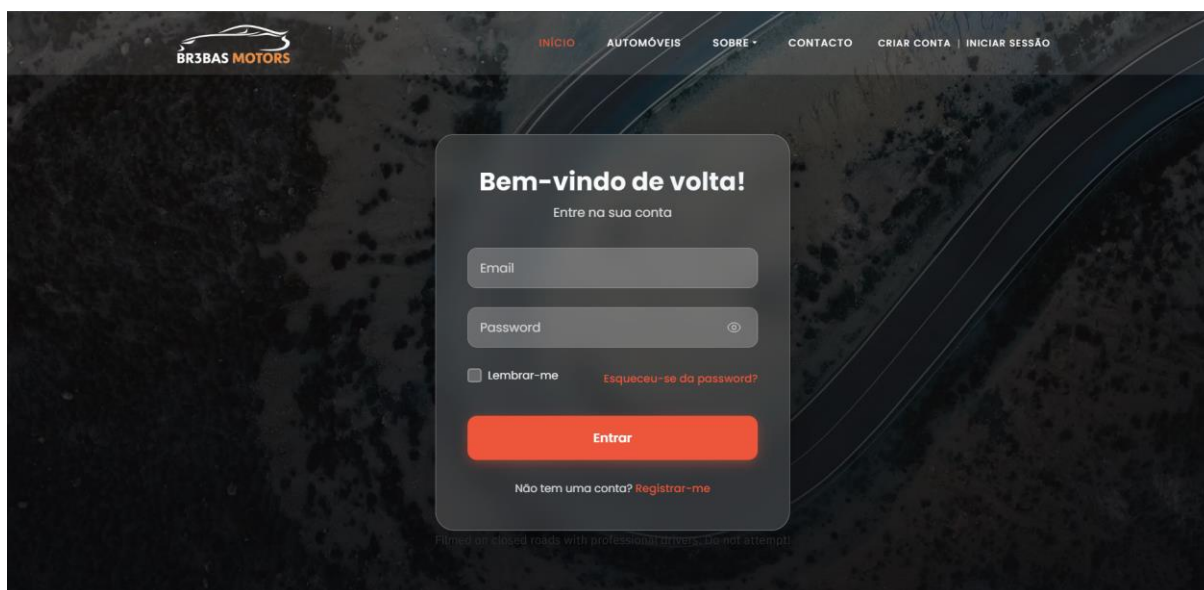


Figura 21 - Página de Login

Por fim, o BackOfficeController organiza as funcionalidades administrativas, permitindo listar utilizadores, atualizar cargos e gerir permissões. Este controlador assegura que apenas administradores têm acesso às operações sensíveis, mantendo a integridade e segurança da área interna da plataforma.

ID	Marca	Modelo	Ano	Preço	Disponibilidade	Reserva	Estado	Ações
24	Aston Martin	DB11	2022	160.000 EUR	Disponível	Reservar	Usado	Ver Editar Apagar
23	Opel	Corso	2002	2.500 EUR	Vendido	Reservar	Usado	Ver Editar Apagar
22	Peugeot	106 Rallye	2000	12.000 EUR	Disponível	Reservar	Usado	Ver Editar Apagar
7	Porsche	992.2 Turbo S	2026	397.475 EUR	Vendido	Reservar	Novo	Ver Editar Apagar
6	BMW	F82 M4	2019	75.900 EUR	Disponível	Reservar	Usado	Ver Editar Apagar
5	Mercedes-Benz	GLE 350d	2021	90.000 EUR	Disponível	Reservar	Usado	Ver Editar Apagar
4	Audi	RS6 Avant	2024	190.000 EUR	Disponível	Reservar	Novo	Ver Editar Apagar
3	Porsche	991.2 GT3 RS	2018	224.779 EUR	Disponível	Reservar	Usado	Ver Editar Apagar
2	Porsche	992.1 Turbo S	2024	299.599 EUR	Disponível	Reservar	Usado	Ver Editar Apagar
1	BMW	F82 M4 CS	2025	207.400 EUR	Disponível	Reservar	Novo	Ver Editar Apagar

Figura 22 - Backoffice (Gestão do Catálogo de Veículos)

3.11 VIEWS

As interfaces da BrebasMotors foram desenvolvidas utilizando o Blade, o motor de templates do Laravel, que permitiu construir páginas organizadas, reutilizáveis e visualmente consistentes em todas as áreas da aplicação, incluindo o front-end público, a área de utilizador e o BackOffice. Desde o início, foi definida uma estrutura clara para os templates, garantindo que cada secção da plataforma possuía o seu próprio conjunto de ficheiros, mantendo a organização e facilitando a manutenção futura.

A aplicação foi estruturada com dois layouts principais: *app.blade.php*, responsável por suportar toda a interface pública, e *back-admin.blade.php*, dedicado ao BackOffice. Ambos os layouts forneceram a estrutura base das páginas, incluindo o cabeçalho, o rodapé, as secções de conteúdo e os scripts comuns. Esta abordagem permitiu que todas as páginas partilhassem elementos essenciais, assegurando uma identidade visual uniforme e reduzindo a duplicação de código.

Para complementar os layouts, foram criados diversos componentes reutilizáveis, como *partials* de navegação, barras laterais, cartões de apresentação e modais. Estes elementos foram incluídos nos templates sempre que necessário, permitindo que funcionalidades repetidas fossem implementadas de forma modular e eficiente. A utilização de *partials* contribuiu significativamente

para a clareza do código, uma vez que cada componente ficou isolado e fácil de atualizar sem afetar outras partes da aplicação.

A navegação da plataforma foi desenvolvida de forma dinâmica, adaptando-se ao estado do utilizador. Quando o utilizador estava autenticado, eram exibidas opções adicionais, como acesso à conta, lista de favoritos e definições pessoais. No caso dos administradores, o menu incluía ainda elementos exclusivos do BackOffice, garantindo que apenas utilizadores com permissões adequadas tinham acesso às ferramentas de gestão. O rodapé foi mantido comum a todas as páginas públicas, apresentando links gerais, informações institucionais e elementos de navegação secundária.

Com esta estrutura de views, a BrebasMotors passou a dispor de uma interface organizada, modular e coerente, facilitando tanto a experiência de navegação dos utilizadores como o trabalho de manutenção e evolução da plataforma.

3.12 GESTÃO DAS IMAGENS

A gestão das imagens constituiu uma parte essencial do desenvolvimento da BrebasMotors, uma vez que a apresentação visual dos veículos desempenha um papel determinante na experiência do utilizador. Para garantir um sistema completo e funcional, implementei um conjunto de mecanismos que permitiram realizar o upload, armazenamento e apresentação das imagens de forma organizada e eficiente.

O processo de upload foi desenvolvido para suportar múltiplas imagens tanto na criação como na edição de veículos. O formulário disponibilizado ao administrador incluiu funcionalidades de pré-visualização, permitindo verificar as imagens antes de serem guardadas, bem como a possibilidade de ordenar a galeria de acordo com a relevância ou preferência. Esta abordagem assegurou que cada veículo pudesse ser apresentado de forma apelativa e estruturada, com uma imagem principal e uma galeria complementar.

As imagens foram armazenadas no disco *public*, sendo os caminhos relativos guardados diretamente no modelo correspondente. Para garantir que as imagens ficassem acessíveis ao público, foi necessário criar o link simbólico através do comando *php artisan storage:link*, que estabeleceu a ligação entre o diretório de armazenamento e a pasta pública da aplicação. A exibição das imagens foi realizada através das funções *asset()* e *Storage::url()*, que permitiram gerar URLs corretos e compatíveis com o sistema de ficheiros.

A apresentação das imagens foi implementada em diferentes áreas da plataforma. No BackOffice, o administrador dispõe de uma galeria completa, onde pode visualizar todas as imagens associadas ao veículo, reorganizá-las e remover aquelas que desejar. Esta interface foi concebida para facilitar a gestão visual dos automóveis, garantindo que o catálogo se mantém atualizado e bem estruturado. No front-end, as imagens são apresentadas na página individual de cada veículo, com destaque para a imagem principal e uma galeria adicional que permite ao utilizador explorar todos os detalhes visuais do automóvel.

Com este sistema, a BrebasMotors passou a dispor de uma gestão de imagens robusta, intuitiva e totalmente integrada, assegurando que cada veículo seja apresentado de forma profissional e visualmente apelativa, contribuindo para uma experiência de navegação mais completa e envolvente.

3.13 VALIDAÇÃO DOS DADOS

A validação dos dados desempenhou um papel fundamental no desenvolvimento da BrebasMotors, garantindo que toda a informação enviada pelos utilizadores fosse tratada de forma segura, consistente e livre de erros. Desde o início, foi implementado um sistema rigoroso de validação diretamente nos controladores, recorrendo ao método *Request::validate()*, que permitiu aplicar regras específicas a cada campo dos formulários. Estas regras incluíram validações como *required*, *string*, *integer*, *numeric*, bem como verificações dedicadas ao envio de imagens, através de *image*, *mimes* e *max*. Adicionalmente, a regra *exists* assegurou a integridade referencial, garantindo que valores associados a chaves estrangeiras correspondiam efetivamente a registos existentes na base de dados.

Sempre que a validação falhava, o sistema devolvia automaticamente os erros à respetiva view, permitindo que o utilizador recebesse feedback claro e imediato sobre os campos incorretos. Para melhorar a experiência de utilização, os formulários mantinham os valores previamente preenchidos, evitando que o utilizador tivesse de repetir toda a informação e facilitando a correção dos dados inválidos. Esta abordagem contribuiu para uma interação mais intuitiva e eficiente, reduzindo frustrações e prevenindo submissões incorretas.

A validação foi reforçada em operações consideradas críticas, como a criação e edição de veículos, onde campos como preço, ano, quilometragem e imagens exigiam verificações rigorosas para garantir que os dados introduzidos eram coerentes e tecnicamente válidos. Da mesma forma, a atualização de dados pessoais na área de conta, incluindo email e password, foi protegida por regras específicas que asseguraram a segurança das credenciais e a integridade das informações do utilizador.

Além da validação tradicional, operações administrativas sensíveis foram protegidas através de verificações adicionais com *abort_unless*, garantindo que apenas utilizadores autorizados podiam executar determinadas ações. Esta camada extra de segurança foi essencial para impedir acessos indevidos e assegurar que a gestão da plataforma permanecia sob controlo dos administradores.

Com este sistema de validação, a BrebasMotors passou a dispor de um mecanismo robusto e fiável que assegurou a qualidade dos dados, preveniu erros e reforçou a segurança global da aplicação.



3.14 TESTES REALIZADOS

A fase de testes foi essencial para garantir que todas as funcionalidades da BrebasMotors funcionavam corretamente e de forma consistente em diferentes cenários de utilização. Embora o Laravel disponibilize exemplos padrão de testes, a implementação real da plataforma exigiu uma verificação mais aprofundada, especialmente nas áreas que envolvem lógica personalizada e operações críticas.

Os testes das funcionalidades principais incluíram a validação do sistema de autenticação, assegurando que o processo de registo, login e recuperação de palavra-passe funcionava sem falhas e que as permissões eram corretamente aplicadas. O sistema de favoritos foi igualmente testado, verificando se os utilizadores conseguiam adicionar e remover veículos da sua lista sem inconsistências, tanto através de AJAX como por redirecionamento tradicional. A gestão de permissões administrativas foi outra área que exigiu atenção, garantindo que apenas utilizadores com privilégios adequados conseguiam aceder ao BackOffice e executar ações sensíveis, como editar ou remover veículos.

O CRUD de automóveis foi testado de forma exaustiva, incluindo a criação, edição, visualização e eliminação de veículos. Estes testes permitiram identificar pequenos problemas relacionados com a validação de dados, o funcionamento das reservas e a atualização do estado dos automóveis. A gestão de imagens também exigiu testes específicos, assegurando que o upload, a remoção e a ordenação da galeria funcionavam corretamente em diferentes navegadores e dispositivos.

Durante esta fase, foram corrigidos diversos erros que surgiram ao longo do desenvolvimento. Entre os ajustes realizados destacam-se melhorias no funcionamento das reservas, correções no sistema de upload de imagens e ajustes nas notificações enviadas aos utilizadores. Estas correções garantiram que a aplicação funcionasse de forma estável, coerente e alinhada com os requisitos definidos inicialmente.

Com a realização destes testes, a BrebasMotors atingiu um nível elevado de fiabilidade, assegurando que todas as funcionalidades principais estavam prontas para utilização real e que a experiência do utilizador seria fluida e sem interrupções.

4. PROBLEMAS E SOLUÇÕES ENCONTRADAS

Durante o desenvolvimento do projeto surgiram diversos desafios que exigiram capacidade de adaptação e resolução. O principal problema ocorreu logo no início, quando o computador portátil da escola sofreu uma avaria que resultou na perda total do projeto. Esta situação obrigou a recomeçar todo o trabalho do zero, o que implicou um atraso significativo na calendarização e exigiu uma reorganização completa das tarefas.

Outro obstáculo enfrentado foi a dificuldade na instalação do Composer, ferramenta essencial para o funcionamento do Laravel. A configuração inicial apresentou erros de compatibilidade e dependências, o que atrasou o processo de preparação do ambiente de desenvolvimento. Após várias tentativas e pesquisa de soluções, o problema foi resolvido através da reinstalação correta dos pacotes e da verificação das variáveis de ambiente.

Durante a fase de programação, surgiram também complicações relacionadas com os recursos visuais de CSS, JavaScript, fontes de texto e imagens, que inicialmente não estavam a ser aplicados às views do projeto. Este problema ocorreu porque os ficheiros não estavam corretamente ligados às pastas públicas do Laravel. A solução passou por ajustar a estrutura das pastas e atualizar os caminhos relativos, garantindo que todos os elementos visuais fossem corretamente carregados e aplicados ao website.

Apesar das dificuldades, cada problema contribuiu para o desenvolvimento de novas competências técnicas e para uma melhor compreensão do funcionamento das ferramentas utilizadas. A superação destes desafios permitiu concluir o projeto com sucesso e reforçou a importância da persistência e da atenção aos detalhes em cada etapa do processo.

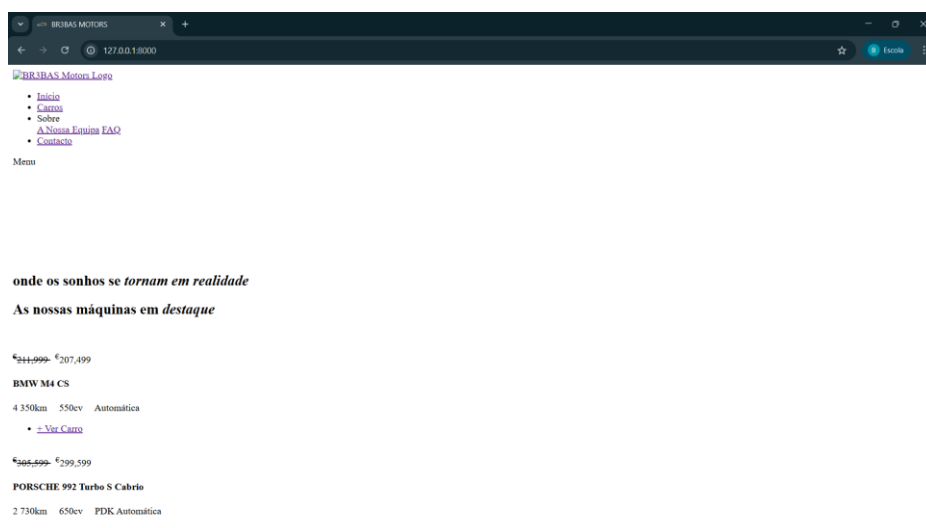


Figura 23 - Problema com os Recursos Visuais do Site

5. CONCLUSÃO

O desenvolvimento da BrebasMotors representou um processo completo de aplicação prática dos conhecimentos adquiridos ao longo da formação, permitindo transformar uma ideia inicial numa plataforma funcional, organizada e alinhada com as necessidades identificadas no início do projeto. A solução final possibilita aos utilizadores pesquisar veículos de forma rápida e personalizada, consultar informação detalhada e guardar favoritos, garantindo uma experiência intuitiva e coerente. Paralelamente, a implementação do BackOffice permitiu criar um ambiente administrativo robusto, onde é possível gerir o catálogo de veículos, atualizar dados, controlar reservas e atribuir permissões, assegurando um funcionamento semelhante ao de uma concessionária real.

Ao longo do desenvolvimento, surgiram desafios técnicos que exigiram análise, adaptação e resolução contínua, desde a estruturação da base de dados até à implementação de funcionalidades mais complexas, como autenticação, permissões, gestão de imagens e validação rigorosa dos dados. Estes momentos foram fundamentais para consolidar competências e aprofundar o domínio das tecnologias utilizadas. Apesar de o projeto estar concluído, existe margem para evolução futura, nomeadamente através da integração de sistemas de comparação de veículos, comunicação direta entre utilizadores e administradores, recolha e análise de estatísticas de utilização ou ligação a APIs externas para atualização automática de dados técnicos.

Relativamente ao meu desempenho, considero que a realização deste projeto foi uma etapa decisiva no meu percurso académico. Consegui cumprir os objetivos definidos, ultrapassar dificuldades e aplicar de forma prática os conhecimentos adquiridos ao longo dos três anos de curso. Este trabalho representa não só a conclusão de um desafio, mas também uma demonstração clara da evolução, autonomia e maturidade técnica que fui desenvolvendo. A BrebasMotors tornou-se, assim, um reflexo do meu crescimento enquanto aluno e futuro profissional.



REFERÊNCIAS BIBLIOGRÁFICAS

Vídeo do Youtube

LARA, C. [laracasts]. (2025, dezembro 3). *Legendas com numeração automática* [Vídeo]. Youtube. <https://youtu.be/SqTdHCTWqks?si=lr4dEP8CFcqN2oYj>

Documentos na Web

BOOTSTRAP. (2025, agosto 25). Bootstrap Documentation. <https://getbootstrap.com/docs/>



TGP SI
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

educação

PESSOAS
2030

PORTUGAL
2030

Cofinanciado pela
União Europeia



SELO DE
CONFORMIDADE
EQAVET

Os Fundos Europeus estão próximos de si.

IDENTIFICAÇÃO DO ALUNO

NOME COMPLETO: BERNARDO ÂNGELO SANTOS

N.º DE PROCESSO: 102186

CICLO DE ESTUDOS: ENSINO SECUNDÁRIO

TELEMÓVEL: 911787311

E-MAIL: 2025A102186@AERP.PT
